

Portafolio Técnico Personal

Documentación Completa del Proyecto

Versión: 1.0

Fecha: Mayo 2026

Autor: marinm80

Repo: github.com/marinm80/technical-portfolio

Stack: React + TypeScript + Tailwind CSS + Vite

Portafolio Técnico Personal — Documentación Completa del Proyecto

Versión: 1.0 Fecha: Mayo 2026 Autor: marinm80 Repositorio:
<https://github.com/marinm80/technical-portfolio>

Tabla de Contenidos

1. [Descripción General](#)
 2. [Arquitectura del Proyecto](#)
 3. [Stack Tecnológico](#)
 4. [Estructura de Archivos](#)
 5. [Fase 1 — Configuración Inicial del Proyecto](#)
 6. [Fase 2 — Sistema de Diseño y Estilos](#)
 7. [Fase 3 — Layout Principal y Navegación](#)
 8. [Fase 4 — Desarrollo de Páginas](#)
 9. [Fase 5 — Internacionalización \(i18n\)](#)
 10. [Fase 6 — Formulario de Contacto](#)
 11. [Fase 7 — CI/CD con GitHub Actions](#)
 12. [Fase 8 — Despliegue a Producción](#)
 13. [Estándares de Ingeniería](#)
 14. [Comandos de Referencia Rápida](#)
 15. [Variables de Entorno](#)
 16. [Roadmap Futuro](#)
-

1. Descripción General

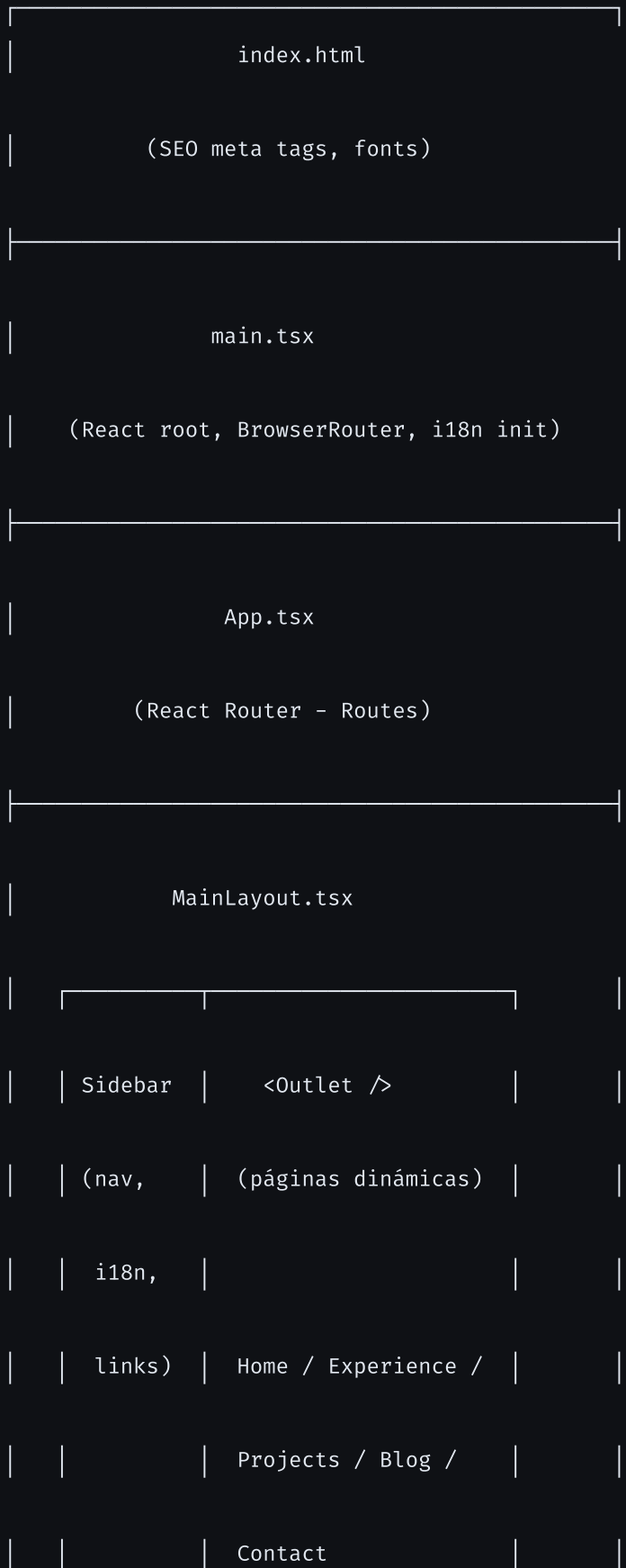
Este proyecto es un **portafolio profesional multilingüe** diseñado como una Single Page Application (SPA). La filosofía central es que el portafolio en sí mismo es un proyecto técnico: demuestra

estándares de ingeniería robustos como TypeScript estricto, validación en tiempo de ejecución con Zod, internacionalización, animaciones de alto rendimiento y CI/CD automatizado.

Características principales:

- **Soporte Multilingüe** (Español / Inglés) con detección automática del idioma del navegador
 - **Tema oscuro corporativo** con paleta obsidiana y acento cyan
 - **Blog técnico** con artículos expandibles
 - **Animaciones fluidas** con Framer Motion
 - **Formulario de contacto** validado con Zod y React Hook Form
 - **CI/CD** con GitHub Actions (lint, type-check, build)
 - **SEO optimizado** con metatags Open Graph y Twitter Cards
-

2. Arquitectura del Proyecto





Patrón: Layout persistente con Sidebar fijo a la izquierda (desktop) o drawer (mobile). Las páginas se renderizan en el `Router` de React Router.

3. Stack Tecnológico

Categoría	Tecnología	Versión	Propósito
Framework	React	19.x	UI Components
Bundler	Vite	8.x	Dev server y build
Lenguaje	TypeScript	6.x	Tipado estático
Estilos	Tailwind CSS	4.x	Utility-first CSS
Animación	Framer Motion	12.x	Transiciones y animaciones
Routing	React Router DOM	7.x	Navegación SPA
i18n	i18next + react-i18next	26.x / 17.x	Internacionalización
Formularios	React Hook Form	7.x	Gestión de formularios
Validación	Zod	4.x	Validación runtime
Íconos	Lucide React	1.x	Biblioteca de íconos
CSS Utils	clsx + tailwind-merge	—	Composición de clases
Linter	ESLint	10.x	Calidad de código
CI/CD	GitHub Actions	—	Automatización

4. Estructura de Archivos

```
technical-portfolio/
├── .github/

|   └── workflows/

|       └── ci.yml                # Pipeline CI/CD

├── frontend/

|   ├── public/                  # Assets estáticos

|   ├── src/

|       ├── assets/              # Imágenes y recursos

|       ├── components/

|           ├── Sidebar.tsx      # Navegación lateral

|           ├── icons/

|               ├── GithubIcon.tsx # Ícono SVG GitHub

|               └── LinkedInIcon.tsx # Ícono SVG LinkedIn

|           ├── layouts/

|               ├── MainLayout.tsx # Layout con sidebar + outlet

|               └── pages/

|                   ├── Home.tsx  # Página principal

|                   ├── Experience.tsx # Timeline de experiencia

|                   └── Projects.tsx # Grid de proyectos
```

```

| | | └─ Blog.tsx          # Artículos técnicos

| | | └─ Contact.tsx      # Formulario de contacto

| | └─ i18n.ts            # Config de internacionalización

| | └─ index.css          # Tema global (Tailwind)

| | └─ App.css            # Estilos legacy (Vite scaffold)

| | └─ App.tsx            # Definición de rutas

| | └─ main.tsx           # Entry point

| └─ index.html           # HTML con SEO metatags

| └─ vite.config.ts       # Config de Vite

| └─ tsconfig.json        # Config TypeScript (raíz)

| └─ tsconfig.app.json    # Config TS para la app

| └─ tsconfig.node.json   # Config TS para Node

| └─ eslint.config.js     # Config ESLint

| └─ package.json         # Dependencias y scripts

└─ docs/                  # Documentación del proyecto

└─ readme.md              # README principal

```

5. Fase 1 — Configuración Inicial del Proyecto

Paso 1: Crear el proyecto con Vite

```
mkdir technical-portfolio && cd technical-portfolio
mkdir frontend && cd frontend

npx -y create-vite@latest ./ --template react-ts

npm install
```

Paso 2: Instalar dependencias del proyecto

```
# Dependencias de producción
npm install react-router-dom framer-motion lucide-react i18next react-i18next react-hook-form @hookform/resolvers



---



npm install -D tailwindcss @tailwindcss/vite @types/node
```


Paso 3: Configurar Vite (vite.config.ts)

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

import tailwindcss from '@tailwindcss/vite'

export default defineConfig({

  plugins: [

    react(),

    tailwindcss(),

  ],

})
```

Paso 4: Configurar TypeScript (tsconfig.app.json)

Puntos clave de la configuración:

- target: "es2023" — JavaScript moderno
- module: "esnext" — Módulos ES nativos
- moduleResolution: "bundler" — Resolución para Vite
- noUnusedLocals: true — Error en variables sin usar
- noUnusedParameters: true — Error en parámetros sin usar
- jsx: "react-jsx" — Transform JSX automático

Paso 5: Inicializar Git

```
cd .. # Volver a la raíz del proyecto
git init

git add .

git commit -m "feat: initial project setup with Vite + React + TypeScript"
```

6. Fase 2 — Sistema de Diseño y Estilos

Paleta de colores (Tema Obsidiana)

Token	Hex	Uso
<code>--color-obsidian</code>	<code>#0f1115</code>	Fondo principal
<code>--color-obsidian-light</code>	<code>#161a22</code>	Fondo de tarjetas/sidebar
<code>--color-obsidian-border</code>	<code>#21262d</code>	Bordes y separadores
<code>--color-accent</code>	<code>#38bdf8</code>	Color de acento (sky-400)

Tipografía

Tipo	Fuente	Uso
Sans-serif	Inter	Texto general, UI
Monospaced	Fira Code	Etiquetas técnicas, código

```
@import "tailwindcss";

@theme {

  --color-obsidian: #0f1115;

  --color-obsidian-light: #161a22;

  --color-obsidian-border: #21262d;

  --color-accent: #38bdf8;

  --font-sans: "Inter", system-ui, sans-serif;

  --font-mono: "Fira Code", monospace;

}

@layer base {

  body {

    @apply bg-obsidian text-slate-300 font-sans antialiased;

  }

  h1, h2, h3, h4, h5, h6 {

    @apply text-slate-50 tracking-tight font-medium;

  }

}
```

Fuentes en `index.html`

```
<link rel="preconnect" href="https://fonts.googleapis.com">
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>

<link href="https://fonts.googleapis.com/css2?family=Fira+Code:wght@400;500;600&family=Inter:wght@400;500;600" rel="stylesheet">
```

7. Fase 3 — Layout Principal y Navegación

7.1 Utilidad `cn()` para clases CSS

```
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";

function cn( ...inputs: ClassValue[] ) {

  return twMerge(clsx(inputs));

}
```

Esta función combina `clsx` (condicionales de clases) con `tailwind-merge` (resuelve conflictos de Tailwind).

7.2 MainLayout (`layouts/MainLayout.tsx`)

Responsabilidad: Renderizar el Sidebar fijo y el área de contenido principal.

- En desktop: sidebar fijo de 256px (`w-64`) a la izquierda, contenido con `ml-64`
- En mobile: header fijo con botón hamburguesa, sidebar como drawer con backdrop
- El `Router` de React Router renderiza la página activa

7.3 Sidebar (`components/Sidebar.tsx`)

Responsabilidad: Navegación principal, enlaces sociales, selector de idioma.

Elementos:

- **Header:** Nombre + título profesional
- **Navegación:** Links a Overview, Experience, Projects, Blog (con `NavLink` para estado activo)

- **Footer:** Links a GitHub/LinkedIn, link a Contact, botón de cambio de idioma
- **Mobile:** Botón de cierre (X), backdrop con blur

7.4 Definición de Rutas (`App.tsx`)

```
<Routes>
  <Route element={ <MainLayout /> }>

    <Route path="/" element={ <Home /> } />

    <Route path="/experience" element={ <Experience /> } />

    <Route path="/projects" element={ <Projects /> } />

    <Route path="/blog" element={ <Blog /> } />

    <Route path="/contact" element={ <Contact /> } />

    <Route path="*" element={ <Navigate to="/" replace /> } />

  </Route>

</Routes>
```

7.5 Entry Point (`main.tsx`)

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'

import { BrowserRouter } from 'react-router-dom'

import './index.css'

import './i18n'

import App from './App.tsx'

createRoot(document.getElementById('root')!).render(

  <StrictMode>

    <BrowserRouter>

      <App />

    </BrowserRouter>

  </StrictMode>,

)
```

8. Fase 4 — Desarrollo de Páginas

8.1 Home (/)

Propósito: Primera impresión. Titular impactante, bio concisa, acceso rápido a experiencia y LinkedIn.

Secciones:

1. **Hero:** Título animado con fade-in ("Ingeniero de Software / especializado en React y Node.js")
2. **Bio:** Párrafo corto describiendo la propuesta de valor

3. **CTAs:** Botón "Ver Experiencia" + link a LinkedIn

4. **Stack:** Tags con las tecnologías principales (TypeScript, React, Next.js, Node.js, Tailwind CSS, PostgreSQL, AWS)

Animación: `framer-motion` con `initial={{ opacity: 0, y: 10 }}` → `animate={{ opacity: 1, y: 0 }}`

8.2 Experience (`/experience`)

Propósito: Timeline vertical de experiencia laboral con énfasis en logros técnicos cuantificables.

Estructura de datos:

```
{
  id: number,

  role: string,          // "Senior Frontend Engineer"

  company: string,       // "Tech Corp"

  period: string,        // "2021 – Presente"

  description: string[]  // Array de logros (viñetas)
}
```

Interacción:

- Se muestran los 2 primeros logros por defecto
- Botón "Ver más" / "Ver menos" con animación `AnimatePresence`
- Timeline con línea vertical gradiente y marcadores circulares con borde `accent`

8.3 Projects (`/projects`)

Propósito: Grid de proyectos destacados con stack técnico y enlaces.

Estructura de datos:

```

{
  id: number,

  title: string,

  description: string,

  tags: string[],          // ["Node.js", "Docker", "RabbitMQ"]

  github: string,

  live: string

}

```

Layout: Grid responsivo `grid-cols-1 md:grid-cols-2` con tarjetas que tienen hover en el borde.

8.4 Blog (`/blog`)

Propósito: Artículos técnicos expandibles in-place.

Estructura de datos:

```

{
  id: number,

  title: string,

  date: string,

  readingTime: string,

  summary: string,

  content: string[]        // Párrafos del artículo

}

```

Interacción:

- Click en el título o botón expande/colapsa el contenido completo
- Animación de altura con `AnimatePresence`
- Metadata visible: fecha y tiempo de lectura

8.5 Contact (/contact)

Propósito: Formulario de contacto validado con feedback visual.

Stack del formulario:

- **React Hook Form:** Gestión del estado del formulario
- **Zod:** Esquema de validación runtime
- **@hookform/resolvers:** Integración Zod ↔ React Hook Form

Esquema de validación:

```
const contactSchema = z.object({
  name: z.string().min(2, "El nombre debe tener al menos 2 caracteres."),

  email: z.string().email("Debe ser un email válido."),

  message: z.string().min(10, "El mensaje debe tener al menos 10 caracteres."),

});
```

Flujo:

1. Usuario llena campos con validación en tiempo real
 2. Al enviar: estado `isSubmitting` deshabilita el botón y muestra "Enviando..."
 3. Éxito: se muestra pantalla de confirmación con ícono `CheckCircle` verde
 4. La confirmación desaparece automáticamente después de 5 segundos
-

9. Fase 5 — Internacionalización (i18n)

Configuración (`i18n.ts`)

```
import i18n from 'i18next';
import { initReactI18next } from 'react-i18next';

const resources = {

  en: {

    translation: {

      "nav.overview": "Overview",

      "nav.experience": "Experience",

      // ... más claves

    }

  },

  es: {

    translation: {

      "nav.overview": "Resumen",

      "nav.experience": "Experiencia",

      // ... más claves

    }

  }

};
```

```
i18n.use(initReactI18next).init({

  resources,

  lng: 'es',           // Idioma por defecto

  fallbackLng: 'en',   // Fallback

  interpolation: { escapeValue: false }

});
```

Uso en componentes

```
const { t, i18n } = useTranslation();

// Leer traducción

<h1>{t('home.title')}</h1>

// Cambiar idioma

const toggleLanguage = () => {

  const newLang = i18n.language === 'es' ? 'en' : 'es';

  i18n.changeLanguage(newLang);

};
```

10. Fase 6 — Formulario de Contacto

Integración con EmailJS (producción)

Para conectar con EmailJS en producción:

1. Crear cuenta en emailjs.com
2. Configurar un servicio de email (Gmail, Outlook, etc.)
3. Crear una plantilla de email
4. Obtener las credenciales (Service ID, Template ID, Public Key)
5. Configurar variables de entorno:

```
VITE_EMAILJS_SERVICE_ID=tu_service_id  
VITE_EMAILJS_TEMPLATE_ID=tu_template_id  
  
VITE_EMAILJS_PUBLIC_KEY=tu_public_key
```

6. Instalar el SDK: `npm install @emailjs/browser`
7. Reemplazar el mock en `Contact.tsx` :

```
import emailjs from '@emailjs/browser';

const onSubmit = async (data: ContactFormValues) => {

  try {

    await emailjs.send(

      import.meta.env.VITE_EMAILJS_SERVICE_ID,

      import.meta.env.VITE_EMAILJS_TEMPLATE_ID,

      { from_name: data.name, reply_to: data.email, message: data.message },

      import.meta.env.VITE_EMAILJS_PUBLIC_KEY

    );

    setIsSuccess(true);

    reset();

  } catch (error: unknown) {

    if (error instanceof Error) {

      console.error("EmailJS Error:", error.message);

    }

  }

};
```

11. Fase 7 — CI/CD con GitHub Actions

Archivo `.github/workflows/ci.yml`

```
name: CI/CD Pipeline

on:

  push:

    branches: [ main ]

  pull_request:

    branches: [ main ]

jobs:

  build-and-test:

    runs-on: ubuntu-latest

    defaults:

      run:

        working-directory: ./frontend

    steps:

      - uses: actions/checkout@v4

      - name: Use Node.js 20.x

        uses: actions/setup-node@v4

        with:
```

```
node-version: '20.x'

cache: 'npm'

cache-dependency-path: ./frontend/package-lock.json

- name: Install dependencies

  run: npm ci

- name: Run Linter

  run: npm run lint

- name: Type Checking

  run: npx tsc --noEmit

- name: Build Project

  run: npm run build
```

¿Qué verifica el pipeline?

Paso	Comando	Propósito
Lint	<code>npm run lint</code>	Estilo y errores de código (ESLint)
Types	<code>npx tsc --noEmit</code>	Errores de tipado TypeScript
Build	<code>npm run build</code>	Compilación exitosa para producción

12. Fase 8 — Despliegue a Producción

Opción A: Vercel (recomendado para SPAs)

1. Conectar repositorio en vercel.com
2. Configurar:
 - **Root Directory:** `frontend`
 - **Build Command:** `npm run build`
 - **Output Directory:** `dist`
3. Añadir variables de entorno de EmailJS
4. Deploy automático en cada push a `main`

Opción B: Netlify

1. Conectar repositorio en netlify.com
2. Configurar:
 - **Base directory:** `frontend`
 - **Build command:** `npm run build`
 - **Publish directory:** `frontend/dist`
3. Agregar archivo `_redirects` en `public/`: `/* /index.html 200`

Opción C: Render

1. Crear Static Site en render.com
2. Configurar:
 - **Root Directory:** `frontend`
 - **Build Command:** `npm install && npm run build`
 - **Publish Directory:** `dist`

13. Estándares de Ingeniería

TypeScript Estricto

- `noUnusedLocals: true` — Variables sin usar = error de compilación
- `noUnusedParameters: true` — Parámetros sin usar = error
- Sin `any` — Usar `unknown` + validación Zod para datos externos

Cláusulas de Guarda (Early Returns)

```
// ❌ Mal: if anidados
if (user) {

    if (user.isActive) {

        // lógica

    }

}

// ✅ Bien: early returns

if (!user) return null;

if (!user.isActive) return <Inactive />;

// lógica principal
```

Gestión Defensiva de Errores

```
try {
    const result = await riskyOperation();

} catch (error: unknown) {

    if (error instanceof Error) {

        showError(error.message);

    }

}
```

ESLint Configuración

- `js.configs.recommended` — Reglas base de JavaScript
 - `tsestlint.configs.recommended` — Reglas de TypeScript
 - `reactHooks.configs.flat.recommended` — Reglas de hooks de React
 - `reactRefresh.configs.vite` — Compatibilidad con HMR de Vite
-

14. Comandos de Referencia Rápida

```
# Desarrollo (desde frontend/)
npm run dev           # Servidor de desarrollo (HMR)

npm run build         # Build de producción

npm run preview       # Preview del build

npm run lint          # Ejecutar ESLint
```

```
git add .
```

```
git commit -m "feat: descripción del cambio"
```

```
git push origin main
```

```
npx tsc --noEmit
```

IMPORTANTE: El script `npm run dev` debe ejecutarse desde la carpeta `frontend/`, no desde la raíz del proyecto.

```
cd frontend
npm run dev
```

15. Variables de Entorno

Crear archivo `frontend/.env.local` :

```
# EmailJS (formulario de contacto)
VITE_EMAILJS_SERVICE_ID=tu_service_id_aqui

VITE_EMAILJS_TEMPLATE_ID=tu_template_id_aqui

VITE_EMAILJS_PUBLIC_KEY=tu_public_key_aqui
```

***Nota:** Las variables con prefijo `VITE_` son expuestas al código del cliente. Nunca colocar secretos sensibles aquí.*

16. Roadmap Futuro

- ☐ **Blog con MDX:** Migrar artículos a archivos `.mdx` con resaltado de sintaxis
- ☐ **Tema Claro/Oscuro:** Implementar toggle con React Context y persistencia en localStorage
- ☐ **Testing:** Vitest para unit tests, React Testing Library para componentes
- ☐ **SEO dinámico:** Integrar `react-helmet` para metatags por página
- ☐ **CV descargable:** Añadir PDF en `public/cv.pdf` con botón de descarga
- ☐ **Despliegue a producción:** Deploy en Vercel/Netlify/Render

Documento generado como referencia completa para reconstruir el proyecto desde cero.